Page de garde

AP1 – MediatekFormation

Sarita Nasimiyu

1. Contexte général

1.1 Contexte

L'atelier **AP1 - MediatekFormation** s'inscrit dans un contexte de reprise et d'évolution d'une application web existante. Cette application a pour objectif de permettre la consultation de formations vidéo organisées en playlists, avec une partie publique accessible aux utilisateurs et une partie administration réservée à la gestion du contenu.

Le travail demandé dans le cadre de cet atelier consistait à intervenir sur une base applicative déjà fournie, à la faire évoluer, à corriger certains points techniques, à mettre en place des tests, à documenter les fonctionnalités développées, puis à assurer sa mise en ligne dans un environnement de production.

1.2 Existant

L'application d'origine permettait déjà d'afficher des formations et des playlists, avec une séparation entre la partie visible par les utilisateurs et la partie administration.
Le projet reposait sur une architecture Symfony avec une base de données relationnelle et une interface web dynamique.

1.3 Demande

La demande portait sur plusieurs aspects complémentaires du cycle de vie d'une application :

- Nettoyage et amélioration du code existant ;
- Développement de la partie back-office ;
- Contrôle du bon fonctionnement par les tests ;
- Rédaction et mise à jour de la documentation ;
- Déploiement de l'application et de la base de données ;
- Mise en place d'une sauvegarde de la base ;
- Automatisation du déploiement à chaque mise à jour du dépôt.

L'ensemble de ces travaux devait aboutir à une application fonctionnelle, documentée, déployée en ligne et exploitable dans un cadre réel.

1.4 Langages et technologies utilisés

Les principaux langages et technologies utilisés dans le cadre de cet atelier sont les suivants :

- **PHP**
- **Symfony**
- **Twig**
- **HTML**
- **CSS**
- **Bootstrap**
- **MySQL**
- **Doctrine**
- **Git**
- **GitHub**
- **Railway**
- **MySQL Workbench**
- **Shell script**

2. Sommaire

Page de garde 1
1. Contexte général 2
1.1 Contexte 2
1.2 Existant 2
1.3 Demande 2
1.4 Langages et technologies utilisés 3
2. Sommaire 4
3. Mission 1 – Nettoyer et optimiser le code existant 8
3.1 Tâche 1 – Nettoyer le code existant 8
Demande de la tâche 8
Temps estimé : 2 heures 9
Temps réel : 2 h 15 9
Travail réalisé 9
Étapes suivies 9
Résultat obtenu 9
Captures d'écran 10
3.2 Tâche 2 – Ajouter le nombre de formations par playlist 10
Demande de la tâche 10
Temps estimé : 2 heures 10
Temps réel : 1 h 50 10
Travail réalisé 10
Étapes suivies 11
Résultat obtenu 11
Captures d'écran 11
4. Mission 2 – Coder la partie back-office 12
4.1 Tâche 1 – Gérer les formations 12
Demande de la tâche 12
Temps estimé: 5 heures 13
Temps réel: 7 heures 13

Travail réalisé 13
Étapes suivies 13
Résultat obtenu 14
Captures d'écran 14
4.2 Tâche 2 – Gérer les playlists 15
Demande de la tâche 15
Temps estimé : 5 heures 15
Temps réel: 4 heures 15
Travail réalisé 15
Étapes suivies 16
Résultat obtenu 16
Captures d'écran 17
4.3 Tâche 3 – Gérer les catégories 17
Demande de la tâche 17
Temps estimé: 3 heures 18
Temps réel: 2 h 45 18
Travail réalisé 18
Étapes suivies 18
Résultat obtenu 18
Captures d'écran 19
4.4 Tâche 4 – Ajouter l'accès avec authentification 19
Demande de la tâche 19
Temps estimé: 4 heures 20
Temps réel: 5 heures 20
Travail réalisé 20
Étapes suivies 20
Résultat obtenu 21
Captures d'écran 21
5. Mission 3 – Tester et documenter 22
5.1 Tâche 1 – Gérer les tests 22
Demande de la tâche 22
Temps estimé : 7 heures 22

Temps réel : 10 heures 22
Travail réalisé 22
Étapes suivies 23
Résultat obtenu 23
Captures d'écran 23
5.2 Tâche 2 – Créer la documentation technique 24
Demande de la tâche 24
Temps estimé : 1 heure 25
Temps réel : 2 heures 25
Travail réalisé 25
Étapes suivies 25
Résultat obtenu 25
Captures d'écran 26
5.3 Tâche 3 – Créer la documentation utilisateur 26
Demande de la tâche 26
Temps estimé: 2 heures 27
Temps réel: 1 h 45 27
Travail réalisé 27
Étapes suivies 27
Résultat obtenu 27
Captures d'écran 28
6. Mission 4 – Déploiement, sauvegarde et déploiement continu 29
6.1 Tâche 1 – Déployer le site 29
Demande de la tâche 29
Temps estimé : 2 heures 29
Temps réel : 3 heures 29
Travail réalisé 29
Étapes suivies 29
Résultat obtenu 30
Captures d'écran 30
6.2 Tâche 2 – Gérer la sauvegarde et la restauration de la BDD 31
Demande de la tâche 31

Temps estimé : 1 heure 32
Temps réel : 1 h 05 min 32
Travail réalisé 32
Étapes suivies 32
Organisation de la sauvegarde 33
Mode opératoire pour la restauration 33
Résultat obtenu 34
Captures d'écran 34
6.3 Tâche 3 – Mettre en place le déploiement continu 35
Demande de la tâche 35
Temps estimé : 1 heure 35
Temps réel : 10 minutes 35
Travail réalisé 35
Étapes suivies 35
Résultat obtenu 36
Captures d'écran 36
7. Bilan final 37
7.1 Objectifs atteints 37
7.2 Problèmes rencontrés 37
7.3 Solutions apportées 37
7.4 Compétences mobilisées 38
7.5 Conclusion générale 38

1. Contexte général

1.1 Contexte
1.2 Existant
1.3 Demande
1.4 Langages et technologies utilisés

2. Sommaire

3. Mission 1 – Nettoyer et optimiser le code existant

3.1 Tâche 1 – Nettoyer le code

3.2 Tâche 2 – Ajouter une fonctionnalité

4. Mission 2 – Coder la partie back-office
4.1 Tâche 1 – Gérer les formations
4.2 Tâche 2 – Gérer les playlists
4.3 Tâche 3 – Gérer les catégories
4.4 Tâche 4 – Ajouter l'accès avec authentification

5. Mission 3 – Tester et documenter
5.1 Tâche 1 – Gérer les tests
5.2 Tâche 2 – Créer la documentation technique
5.3 Tâche 3 – Créer la documentation utilisateur

6. Mission 4 – Déploiement, sauvegarde et déploiement continu
6.1 Tâche 1 – Déployer le site
6.2 Tâche 2 – Gérer la sauvegarde et la restauration de la BDD
6.3 Tâche 3 – Mettre en place le déploiement continu

7. Bilan final

8. Annexes

3. Mission 1 – Nettoyer et optimiser le code existant

3.1 Tâche 1 – Nettoyer le code existant

Demande de la tâche

Cette tâche consistait à nettoyer le code créé par le développeur en suivant les recommandations de **SonarLint / SonarQube**, sans modifier le code généré automatiquement par Symfony.

Temps estimé : 2 heures

Temps réel : 2 h 15

Travail réalisé

Le nettoyage du code a été réalisé en s'appuyant sur les recommandations remontées par les outils d'analyse de qualité.
L'objectif était d'améliorer la lisibilité, l'homogénéité, la maintenabilité et l'accessibilité du projet, sans changer son comportement fonctionnel.

Les principales corrections effectuées ont porté sur :

- la suppression des espaces inutiles ;
- l'ajout explicite de certaines visibilités ;
- le remplacement de chaînes dupliquées par des constantes ;
- le remplacement d'un `switch` par une structure conditionnelle plus lisible ;
- le renommage d'une constante afin de respecter les conventions ;
- la fusion de conditions imbriquées ;
- l'ajout d'accolades sur certains blocs ;
- le remplacement des tabulations par des espaces ;
- l'ajout d'attributs `alt` sur les images ;
- l'amélioration de l'accessibilité de certains liens ;
- l'ajout d'une `caption` sur les tables ;
- la correction d'une faute de frappe.

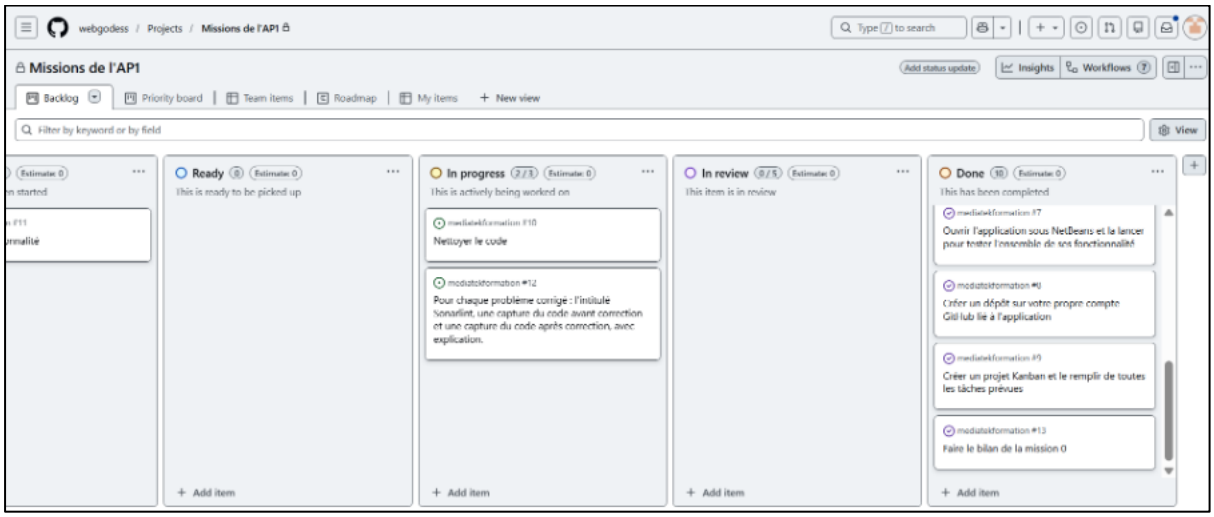
Étapes suivies

Le travail a d'abord consisté à repérer les alertes signalées par **SonarLint / SonarQube** dans les différents fichiers du projet.
Les corrections ont ensuite été appliquées progressivement dans le code source, en veillant à ne pas modifier les fichiers générés automatiquement par Symfony.
Une vérification a enfin été réalisée afin de s'assurer que le nettoyage n'avait pas d'impact sur le fonctionnement de l'application.

Résultat obtenu

Le nettoyage n'a pas modifié le fonctionnement du site, mais il a permis d'obtenir un code plus propre, plus homogène et plus maintenable.
Il a également amélioré l'accessibilité HTML de plusieurs pages et préparé le projet à la suite des développements.

Captures d'écran



3.2 Tâche 2 – Ajouter le nombre de formations par playlist

Demande de la tâche

Cette tâche consistait à ajouter dans la page **Playlists** une colonne affichant le nombre de formations par playlist, un tri croissant et décroissant sur cette colonne, ainsi que l'affichage de cette information dans la page de détail d'une playlist.

Temps estimé : 2 heures

Temps réel : 1 h 50

Travail réalisé

Cette évolution a nécessité plusieurs modifications dans le projet afin de calculer, récupérer et afficher le nombre de formations par playlist.

Une méthode a d'abord été ajoutée dans l'entité `Playlist` afin de retourner le nombre de formations associées à une playlist.
Le repository a ensuite été adapté pour permettre le tri des playlists selon ce nombre.
Le contrôleur a été modifié pour prendre en compte ce nouveau critère de tri, puis les vues Twig ont été mises à jour afin d'afficher cette information dans l'interface.

Étapes suivies

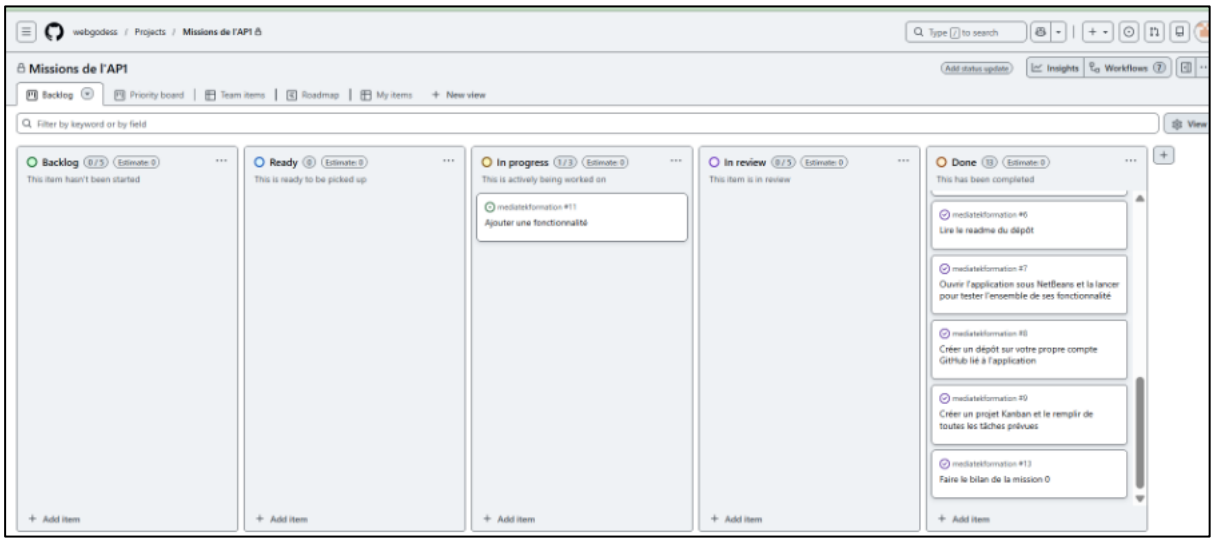
Le travail a été réalisé en plusieurs étapes :

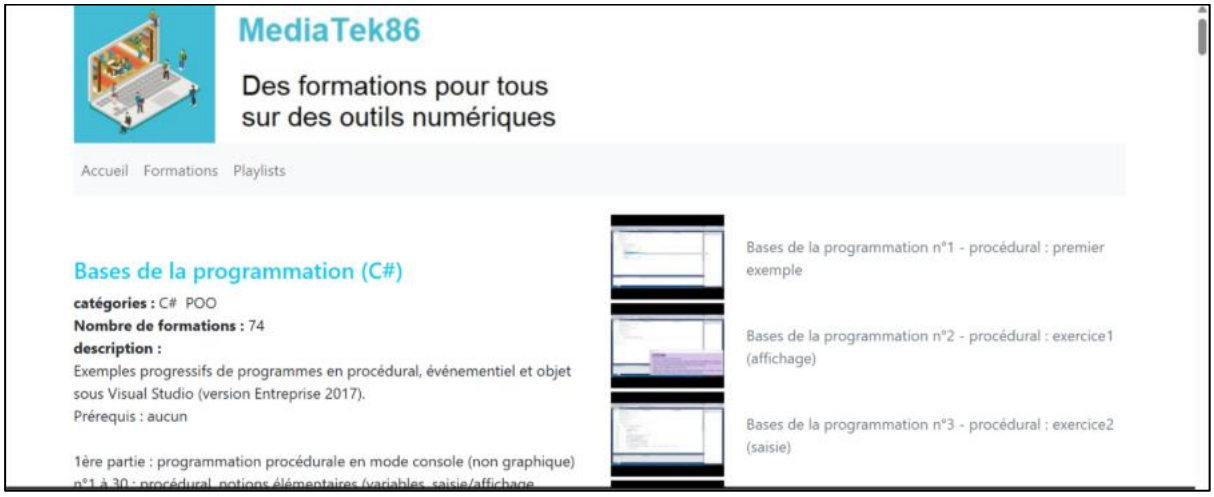
- ajout d'une méthode `getNombreFormations()` dans l'entité `Playlist` afin de compter les formations associées ;
- création d'une méthode spécifique dans le repository pour récupérer les playlists triées selon leur nombre de formations ;
- adaptation de la méthode `sort()` dans le contrôleur afin de gérer ce nouveau cas de tri ;
- modification de la page listant les playlists pour ajouter une colonne « Nombre de formations » avec deux liens de tri ;
- modification de la page de détail d'une playlist afin d'afficher également cette information.

Résultat obtenu

Le nombre de formations est désormais affiché pour chaque playlist.
Cette information apparaît dans la liste des playlists ainsi que dans la page de détail d'une playlist.
Un tri croissant et décroissant sur ce critère a également été ajouté.

Captures d'écran





4. Mission 2 – Coder la partie back-office

4.1 Tâche 1 – Gérer les formations

Demande de la tâche

Cette tâche consistait à mettre en place, dans le back office, une page permettant de lister les formations et d'effectuer sur chacune d'elles des actions de modification ou de suppression.

La page devait également reprendre les mêmes tris et filtres que ceux présents dans le front office.
Un bouton devait permettre d'accéder au formulaire d'ajout d'une formation.

Le formulaire devait contrôler les saisies : la description n'était pas obligatoire, la sélection de catégories pouvait être vide, la playlist devait être choisie dans une liste, les catégories dans

une sélection multiple, et la date devait être sélectionnée sans pouvoir dépasser la date du jour.

Le clic sur le bouton de modification devait ouvrir le même formulaire, mais prérempli avec les données existantes.

Temps estimé: **5 heures**

Temps réel: **7 heures**

Travail réalisé

Une page de back office a été mise en place afin de gérer les formations.
Cette page permet d'afficher les formations sous forme de tableau avec, pour chaque ligne, les informations principales ainsi que les actions disponibles.

La gestion mise en place comprend :

- l'affichage du titre, de la playlist, des catégories et de la date ;
- un bouton **Modifier** ;
- un bouton **Supprimer** ;
- un bouton **Ajouter** permettant d'accéder au formulaire de création.

La suppression d'une formation a également été prévue de manière cohérente avec la base de données, afin qu'une formation supprimée ne reste pas associée à une playlist.

Un formulaire unique a été utilisé pour l'ajout et la modification d'une formation.
Ce formulaire a été configuré avec les contraintes demandées : champs obligatoires ou facultatifs, sélection de la playlist dans une liste, sélection multiple des catégories et contrôle de la date.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

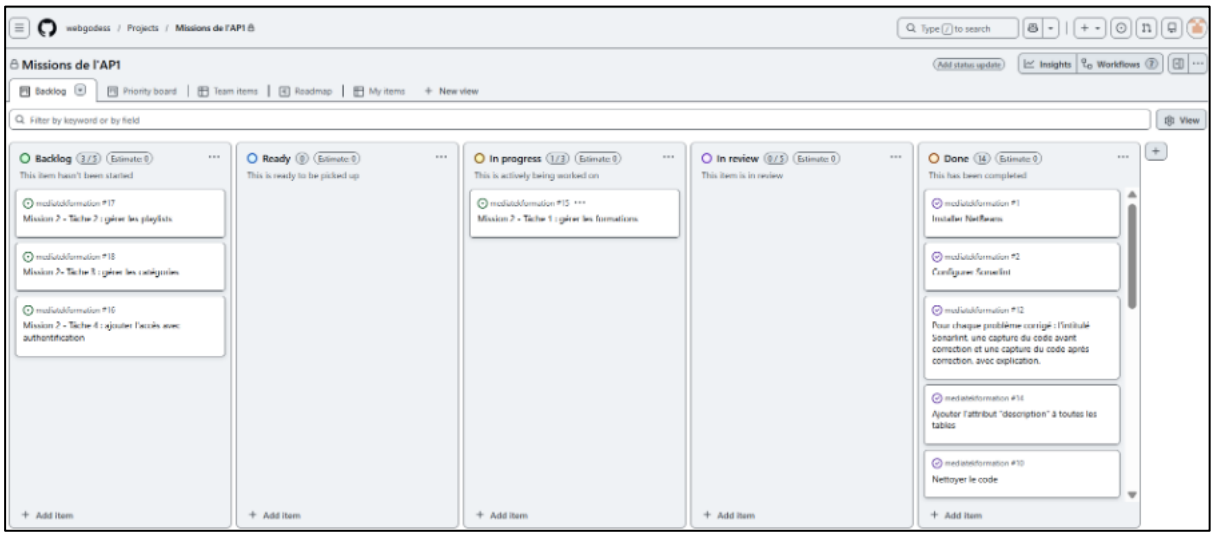
- création ou adaptation du contrôleur d'administration des formations ;
- mise en place de la page listant les formations dans le back office ;
- ajout des tris et filtres déjà présents dans le front office ;
- création du formulaire `FormationType` pour gérer l'ajout et la modification ;
- mise en place du préremplissage du formulaire en mode modification ;
- ajout du mécanisme de suppression avec confirmation ;
- vérification de la cohérence de la mise à jour des données après chaque action.

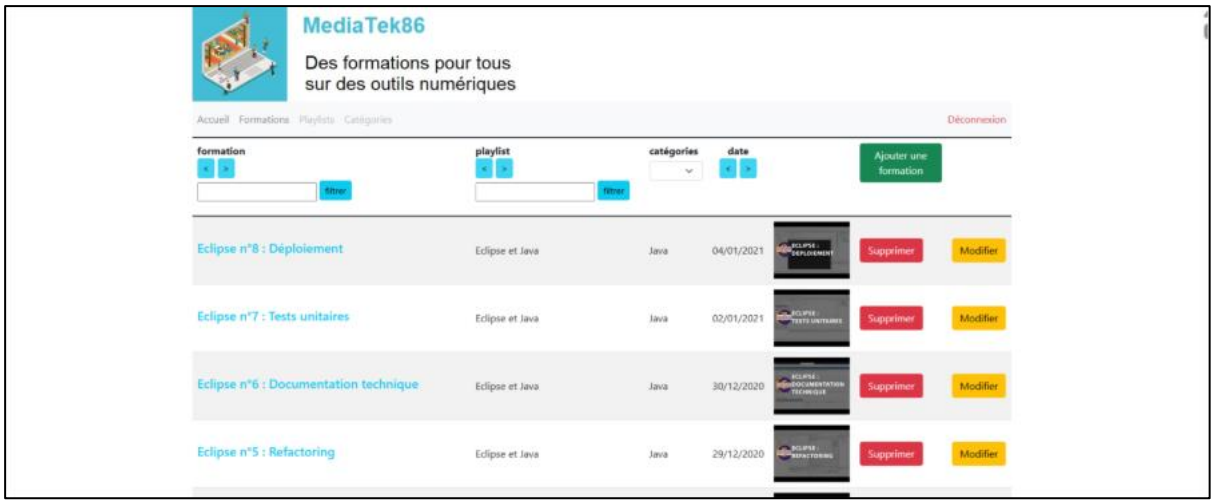
Résultat obtenu

La gestion des formations dans le back office est opérationnelle.
Il est désormais possible d'afficher les formations, de les trier, de les filtrer, d'en ajouter, d'en modifier et d'en supprimer depuis l'interface d'administration.

Le formulaire d'ajout et de modification fonctionne avec les contrôles attendus, et la base de données est correctement mise à jour après chaque action.

Captures d'écran





4.2 Tâche 2 – Gérer les playlists

Demande de la tâche

Cette tâche consistait à mettre en place, dans le back office, la gestion des playlists afin de permettre l'administration du contenu en base de données.

Les fonctionnalités attendues étaient les suivantes :

- afficher la liste des playlists ;
- permettre la modification d'une playlist via un formulaire ;
- permettre la suppression d'une playlist après confirmation, uniquement si aucune formation n'est rattachée ;
- proposer les mêmes tris et filtres que dans le front office ;
- permettre l'ajout d'une playlist via un formulaire avec contrôle des saisies, seul le champ `name` étant obligatoire ;
- afficher, en mode modification, la liste des formations rattachées à la playlist, en lecture seule.

Temps estimé : **5 heures**

Temps réel: **4 heures**

Travail réalisé

Une page de back office a été mise en place afin de gérer les playlists.
Cette page permet d'afficher les playlists dans un tableau avec les informations principales ainsi que les actions disponibles.

La gestion mise en place comprend :

- l'affichage du nom de la playlist ;
- l'affichage du nombre de formations rattachées ;
- l'affichage des catégories associées à partir des formations liées ;
- un bouton **Modifier** ;
- un bouton **Supprimer** ;
- un bouton **Ajouter** permettant d'accéder au formulaire de création.

L'ajout d'une playlist repose sur un formulaire simple avec :

- un champ `name` obligatoire ;
- un champ `description` facultatif.

La modification d'une playlist utilise le même formulaire, mais en mode prérempli.
En complément, la page de modification affiche la liste des formations rattachées à la playlist en lecture seule, sans possibilité de modifier ce rattachement depuis ce formulaire.

La suppression d'une playlist a également été sécurisée : elle nécessite une confirmation et n'est autorisée que si aucune formation n'est encore liée à la playlist.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

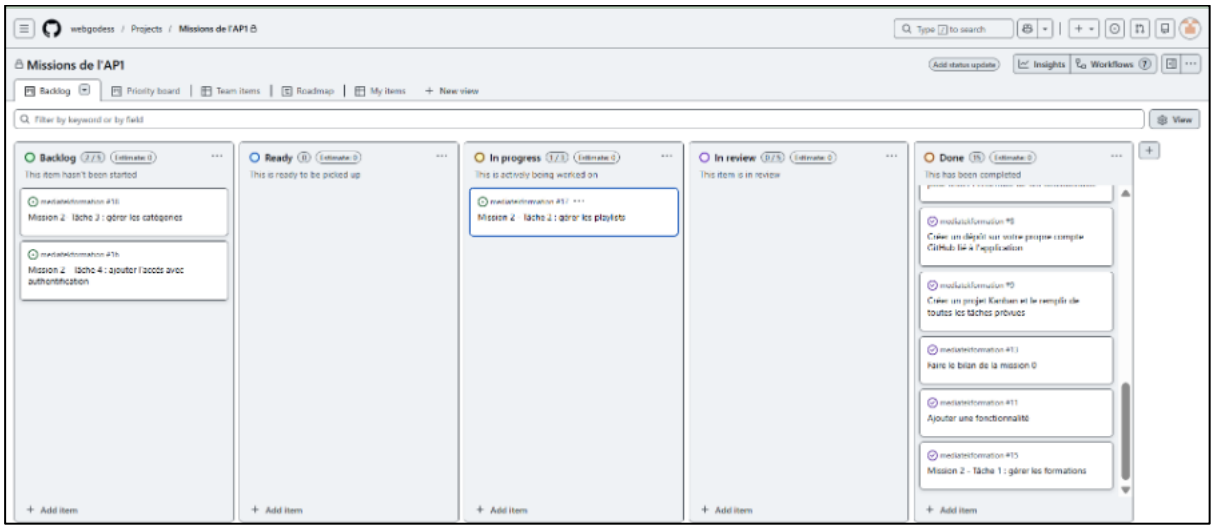
- mise en place de la page de liste des playlists dans le back office ;
- ajout des tris et filtres déjà présents dans le front office ;
- création de la page d'ajout d'une playlist avec validation des saisies ;
- mise en place de la page de modification avec formulaire prérempli ;
- ajout de l'affichage des formations rattachées en lecture seule ;
- mise en place de la suppression avec confirmation ;
- ajout de la règle métier interdisant la suppression d'une playlist contenant encore des formations.

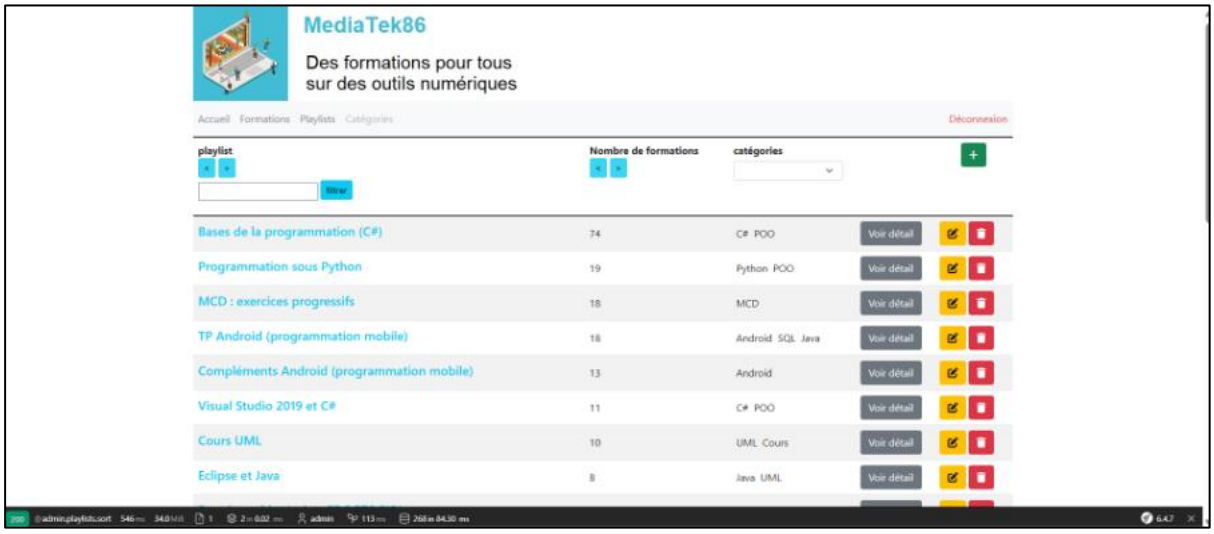
Résultat obtenu

La gestion des playlists dans le back office est opérationnelle.
Il est désormais possible d'afficher les playlists, de les trier, de les filtrer, d'en ajouter, d'en modifier et d'en supprimer depuis l'interface d'administration.

La suppression est sécurisée et respecte la contrainte métier liée aux formations rattachées.
La cohérence entre les playlists et les formations est donc préservée.

Captures d'écran





4.3 Tâche 3 – Gérer les catégories

Demande de la tâche

Cette tâche consistait à mettre en place, dans le back office, la gestion des catégories afin de permettre l'administration du contenu en base de données.

Les fonctionnalités attendues étaient les suivantes :

- afficher la liste des catégories ;
- permettre l'ajout d'une catégorie via un formulaire ;
- permettre la suppression d'une catégorie après confirmation ;
- interdire la suppression si la catégorie est rattachée à au moins une formation ;

- sécuriser les actions à l'aide de requêtes POST et de la protection CSRF.

Temps estimé: 3 heures

Temps réel: 2 h 45

Travail réalisé

Une page de back office a été mise en place afin de gérer les catégories.
Cette page permet d'afficher la liste des catégories sous forme de tableau, avec un formulaire d'ajout et une action de suppression pour chaque ligne.

L'ajout d'une catégorie repose sur un formulaire simple avec un seul champ obligatoire : le nom de la catégorie.
La validation empêche l'enregistrement d'une catégorie vide et garantit l'intégrité des données. La suppression d'une catégorie a également été sécurisée.
Elle nécessite une confirmation, s'effectue via une requête POST protégée par un token CSRF et reste interdite si la catégorie est encore utilisée par au moins une formation.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

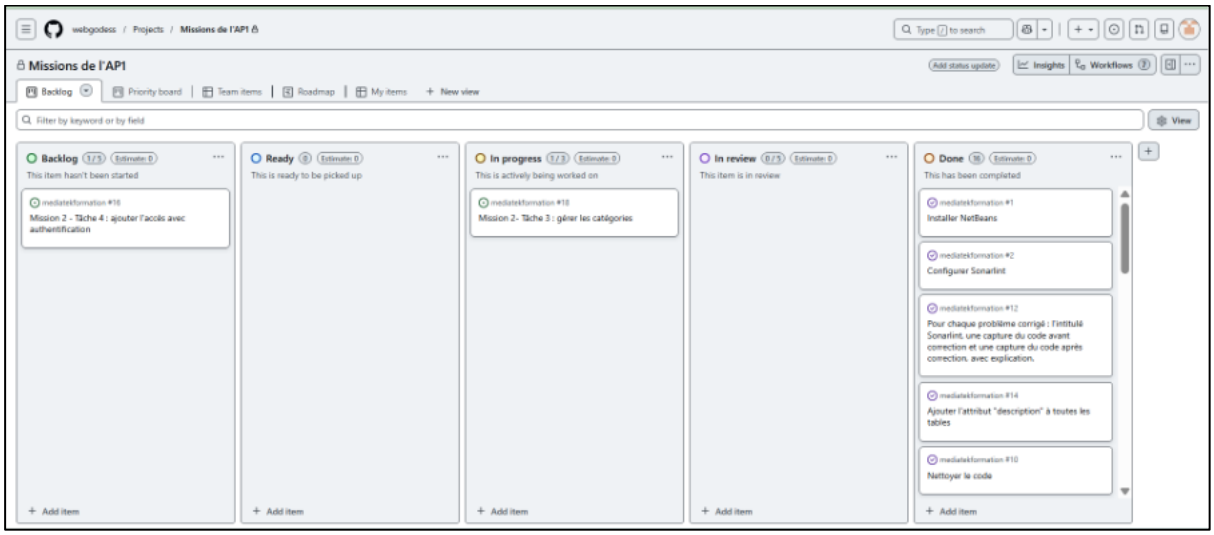
- mise en place de la page de liste des catégories dans le back office ;
- ajout du formulaire d'ajout directement dans cette page ;
- mise en place de la persistance en base via Doctrine ;
- ajout de l'action de suppression avec confirmation ;
- sécurisation de la suppression par requête POST et protection CSRF ;
- ajout d'une vérification empêchant la suppression d'une catégorie encore liée à des formations.

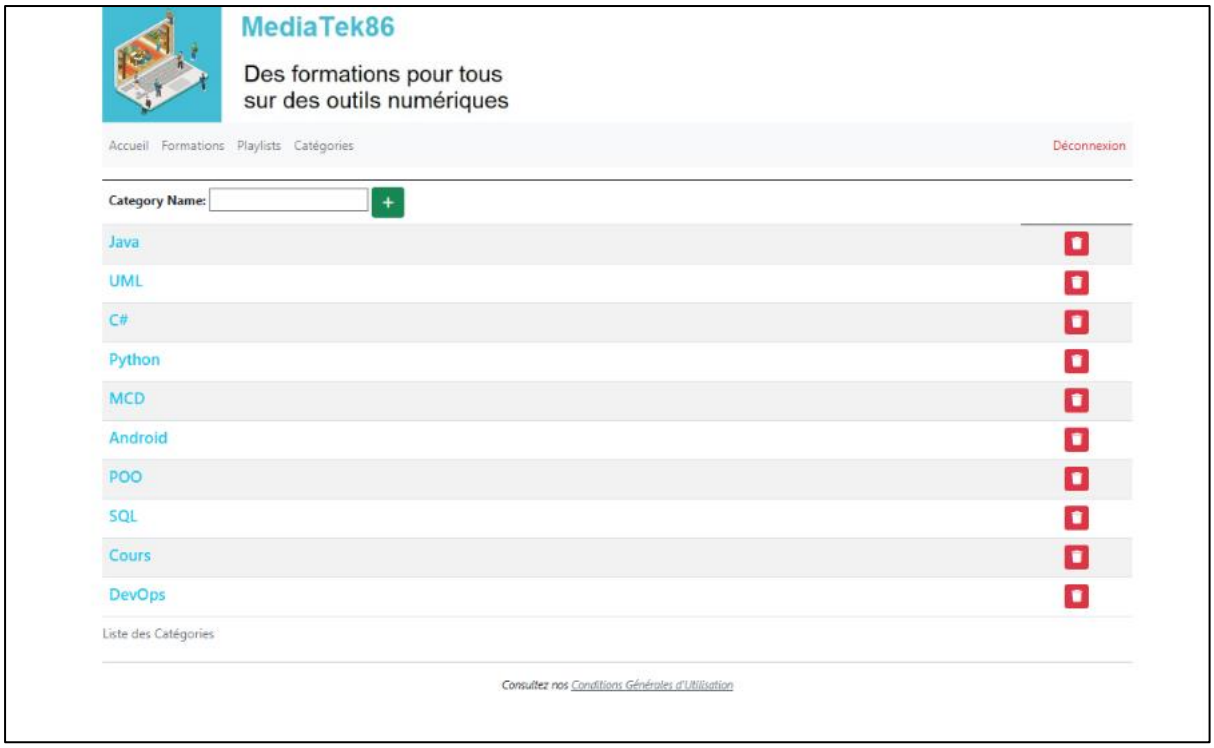
Résultat obtenu

La gestion des catégories dans le back office est opérationnelle.
Il est désormais possible d'afficher les catégories, d'en ajouter et d'en supprimer depuis l'interface d'administration.

La suppression est sécurisée et respecte la contrainte métier interdisant la suppression d'une catégorie encore utilisée.
La cohérence des relations entre catégories et formations est ainsi préservée.

Captures d'écran





4.4 Tâche 4 – Ajouter l'accès avec authentification

Demande de la tâche

Cette tâche consistait à sécuriser l'accès au back office afin que seule une personne disposant du rôle administrateur puisse y accéder.

Les exigences étaient les suivantes :

- le back office doit être accessible uniquement après authentification ;
- un seul profil administrateur doit avoir le droit d'accès ;
- l'accès doit se faire via une URL se terminant par `/admin` ;
- l'accès à toutes les pages du back office doit être sécurisé ;
- un lien de déconnexion doit être présent et fonctionnel sur toutes les pages ;
- la déconnexion doit être opérationnelle.

Temps estimé: 4 heures

Temps réel: 5 heures

Travail réalisé

L'authentification du back office a été mise en place à l'aide du système de sécurité de Symfony.
Un formulaire de connexion a été ajouté afin de permettre à l'administrateur de s'authentifier avant d'accéder aux pages du back office.

L'accès à toutes les routes commençant par `/admin` a ensuite été restreint au rôle `ROLE_ADMIN`.
Un profil administrateur a été prévu en base de données avec un mot de passe hashé et le rôle approprié.

La déconnexion a également été configurée dans Symfony et un lien **Déconnexion** a été ajouté dans les pages du back office afin de permettre la fermeture de session.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

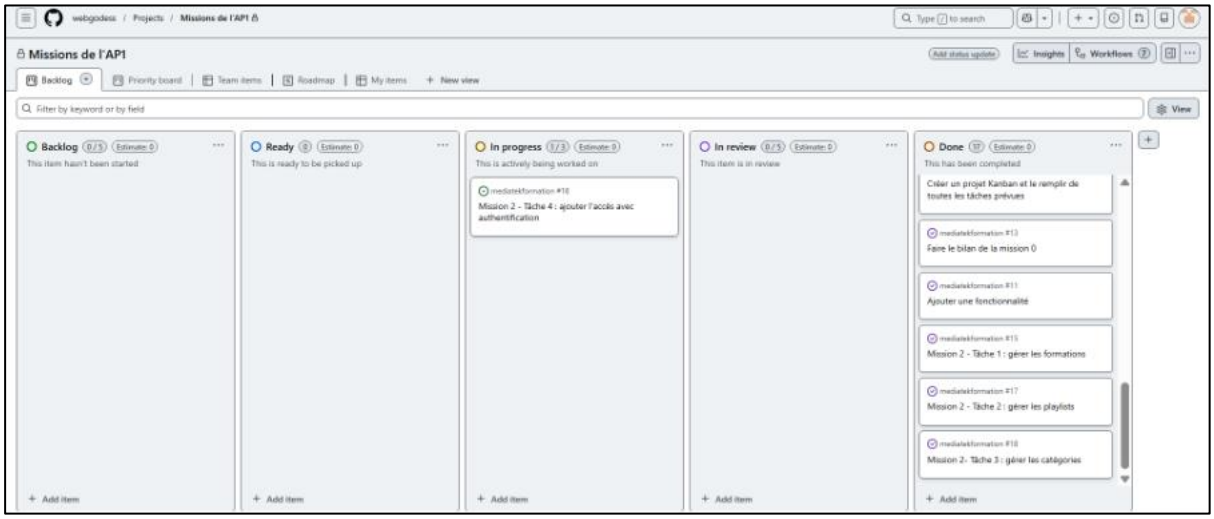
- génération de l'authentification Symfony avec un formulaire de connexion ;
- configuration de la sécurité dans `security.yaml` ;
- ajout d'une règle d'accès restreignant les routes `/admin` au rôle `ROLE_ADMIN` ;
- création ou utilisation d'un compte administrateur en base ;
- ajout du lien de déconnexion dans les pages du back office ;
- vérification de l'accès aux pages d'administration après connexion ;
- vérification du blocage d'accès en l'absence d'authentification.

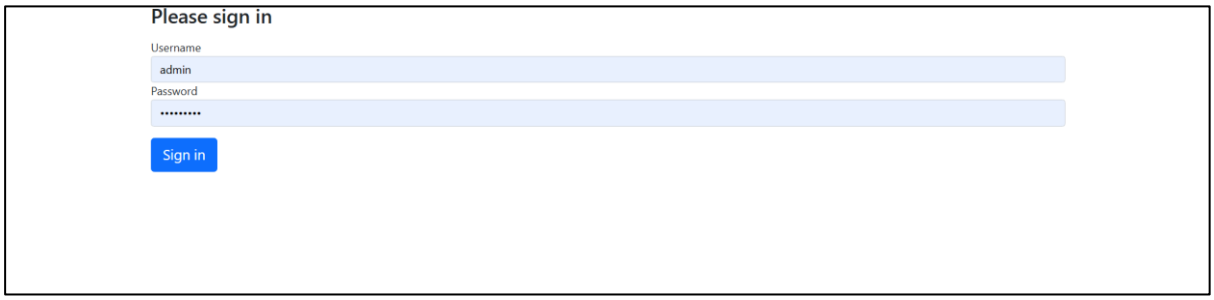
Résultat obtenu

Le back office est désormais sécurisé conformément aux attentes.
Seul un utilisateur disposant du rôle `ROLE_ADMIN` peut accéder aux pages de la zone `/admin`.

La connexion et la déconnexion sont opérationnelles.
Les sessions sont correctement gérées par Symfony et l'accès à la partie administration est protégé.

Captures d'écran





5. Mission 3 – Tester et documenter

5.1 Tâche 1 – Gérer les tests

Demande de la tâche

Cette tâche consistait à vérifier le bon fonctionnement du site à l'aide de plusieurs types de tests.

Les points demandés étaient les suivants :

- un test unitaire sur la méthode retournant la date de parution au format chaîne de caractères ;
- des tests d'intégration sur la validation de la date d'une formation ;
- des tests d'intégration sur les repositories ;
- des tests fonctionnels sur la page d'accueil, les tris, les filtres et les liens des listes ;
- des tests de compatibilité sur plusieurs navigateurs ;
- la rédaction d'un plan de tests ;
- le remplissage du PV de recette puis son enregistrement en PDF.

Temps estimé : **7 heures**

Temps réel : **10 heures**

Travail réalisé

Plusieurs catégories de tests ont été mises en place afin de vérifier le bon fonctionnement de l'application sur les points demandés.

Un test unitaire a permis de contrôler la méthode retournant la date de parution sous forme de chaîne de caractères.
Des tests d'intégration ont ensuite été réalisés pour vérifier la validation de la date d'une formation, notamment l'interdiction d'enregistrer une date postérieure à celle du jour.

Des tests d'intégration ont également été effectués sur les méthodes ajoutées dans les repositories.
En complément, des tests fonctionnels ont permis de vérifier la page d'accueil, les tris, les filtres et les liens présents dans les pages de listes.

Enfin, des tests de compatibilité ont été réalisés sur plusieurs navigateurs afin de confirmer le bon comportement général du site.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

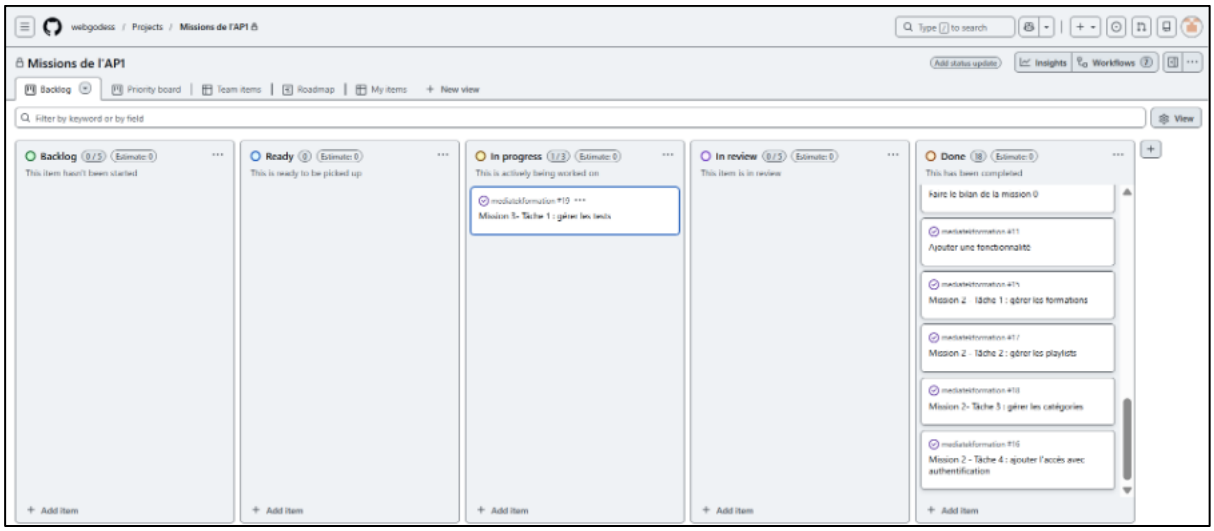
- mise en place d'un test unitaire pour la méthode de formatage de la date ;
- réalisation de tests d'intégration sur les règles de validation des formations ;
- vérification des méthodes de repositories à l'aide d'une base de test ;
- réalisation de tests fonctionnels sur la page d'accueil et les principales pages de listes ;
- vérification du comportement du site sur plusieurs navigateurs ;
- rédaction du plan de tests ;
- remplissage du PV de recette puis export au format PDF.

Résultat obtenu

Les tests réalisés sont conformes aux attentes.
Le bon fonctionnement du site a pu être vérifié sur les points demandés.

Le plan de tests a été rédigé.
Le PV de recette a été complété dans la colonne **« Faite »** puis enregistré au format PDF.

Captures d'écran



```
11  class AdminPlaylistsTest extends WebTestCase
13
    5 references
14  private const PLAYLIST_HOMEPAGE = '/admin/playlists';
    4 references
15  private const PHP_PLAYLIST = 'Playlist PHP';
    8 references
16  private function getAdmin(): KernelBrowser
17  {
18      $client = static::createClient();
19      $adminUser = static::getContainer()
20          ->get(id: UserRepository::class)
21          ->findOneBy(['username' => 'admin']);
22      $client->loginUser(user: $adminUser);
23      return $client;
24  }
25
    0 references | 0 overrides
26  public function testAdminPlaylists(): void
27  {
28      $client = $this->getAdmin();
29      $client->request(method: 'GET', uri: self::PLAYLIST_HOMEPAGE);
30      $this->assertResponseIsSuccessful();
31  }
32
    0 references | 0 overrides
33  public function testAdminPlaylistSort(): void
34  {
35      $client = $this->getAdmin();
36      $client->request(method: 'GET', uri: '/admin/playlists/tri/name/ASC');
37      $this->assertResponseIsSuccessful();
38      $this->assertSelectorTextContains(selector: 'h5', text: 'Playlist Dev Web');
39  }
```

```
62  public function testFormationCategorie()
63      $client = static::createClient();
64      $categorieRepository = static::getContainer()->get(id: CategorieRepository::class);
65      $categorie = $categorieRepository->findOneBy(['name' => 'DevOps']);
66      $categorieId = $categorie->getId();
67      $client->request(method: 'POST', uri: '/formations/recherche/id/categories', parameters: ['recherche' => $categorieId]);
68      $this->assertResponseIsSuccessful();
69      $this->assertCount(expectedCount: 3, haystack: $client->getCrawler()->filter(selector: 'h5'));
70      $this->assertSelectorTextContains(selector: 'h5', text: 'AWS');
71
72
73  }
74
    0 references | 0 overrides
75  public function testFormationShowOne(): void
76  {
77      $client = static::createClient();
78      $formationRepository = static::getContainer()->get(id: FormationRepository::class);
79      $formation = $formationRepository->findOneBy(['title' => 'Symfony']);
80      $formationId = $formation->getId();
81      $client->request(method: 'GET', uri: "/formations/formation/$formationId");
82      $this->assertResponseIsSuccessful();
83      $this->assertSelectorTextContains(selector: 'h4', text: 'Symfony');
84  }
85
86
87
88
89
90  }
91
```

5.2 Tâche 2 – Créer la documentation technique

Demande de la tâche

Cette tâche consistait à produire la documentation technique du site afin de disposer d'une vue structurée du code développé dans le projet.

Les exigences étaient les suivantes :

- contrôler que tous les commentaires normalisés nécessaires à la génération de la documentation technique sont correctement insérés ;
- générer la documentation technique du site complet ;
- inclure les parties front office et back office ;
- exclure le code généré automatiquement par Symfony ;
- exclure le dossier `vendor` ;
- obtenir une documentation portant uniquement sur le code ajouté par les développeurs.

Temps estimé : 1 heure

Temps réel : 2 heures

Travail réalisé

Un contrôle a d'abord été effectué dans les fichiers du projet afin de vérifier la présence des commentaires normalisés nécessaires à la génération de la documentation technique.

Cette étape a permis de s'assurer que les classes, les méthodes importantes et les éléments développés dans le cadre du projet étaient suffisamment commentés pour être correctement pris en compte lors de la génération automatique.

Une fois cette vérification réalisée, la documentation technique du projet a été générée de manière à couvrir les développements du front office et du back office, tout en excluant les parties non demandées.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

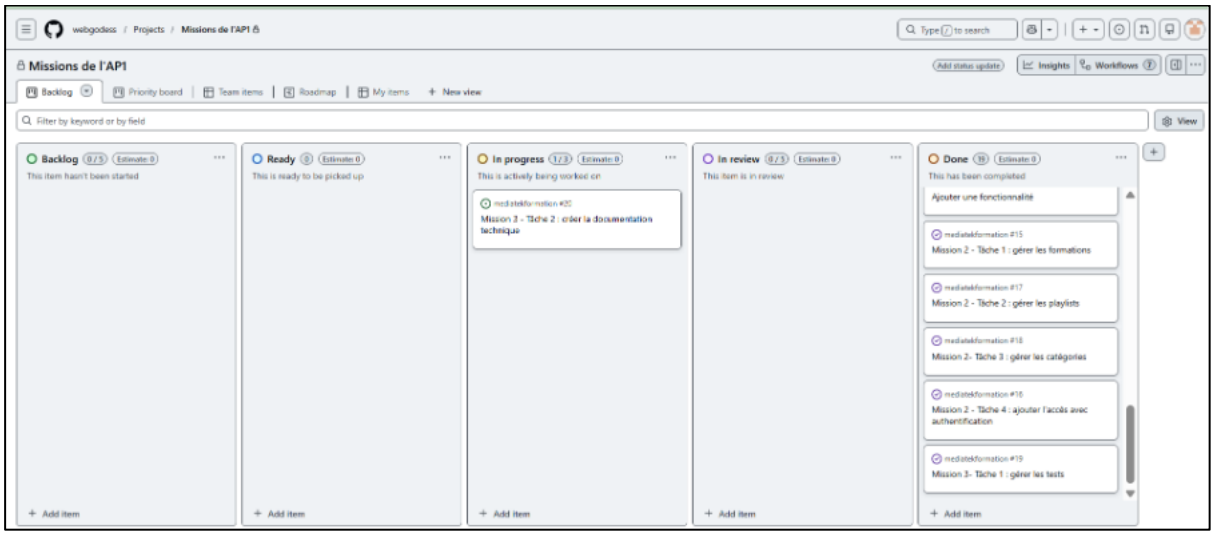
- vérification de la présence des commentaires normalisés dans les fichiers du projet ;
- contrôle de la cohérence des commentaires insérés dans les classes et méthodes principales ;
- paramétrage de la génération de la documentation afin d'exclure le code généré automatiquement par Symfony ;
- exclusion du dossier `vendor` ;
- génération de la documentation technique complète du projet.

Résultat obtenu

La documentation technique du projet a été générée avec succès.
Elle couvre les développements réalisés pour le front office et le back office tout en excluant les fichiers non demandés, notamment le code généré par Symfony et le dossier `vendor`.

La tâche répond donc à la demande initiale et fournit une documentation exploitable pour le suivi technique du projet.

Captures d'écran



5.3 Tâche 3 – Créer la documentation utilisateur

Demande de la tâche

Cette tâche consistait à créer une documentation utilisateur sous forme de vidéo présentant les fonctionnalités du site.

Les exigences étaient les suivantes :

- créer une vidéo montrant les fonctionnalités du front office et du back office ;
- présenter clairement les manipulations réalisées ;
- accompagner les manipulations d'explications orales ;

- respecter une durée maximale de 5 minutes.

Temps estimé: **2 heures**

Temps réel: **1 h 45**

Travail réalisé

Une vidéo de démonstration a été réalisée afin de présenter les principales fonctionnalités du site, aussi bien pour le front office que pour le back office.

La vidéo montre les manipulations à effectuer et les différentes fonctionnalités disponibles, avec des explications orales permettant de guider l'utilisateur dans l'utilisation du site.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

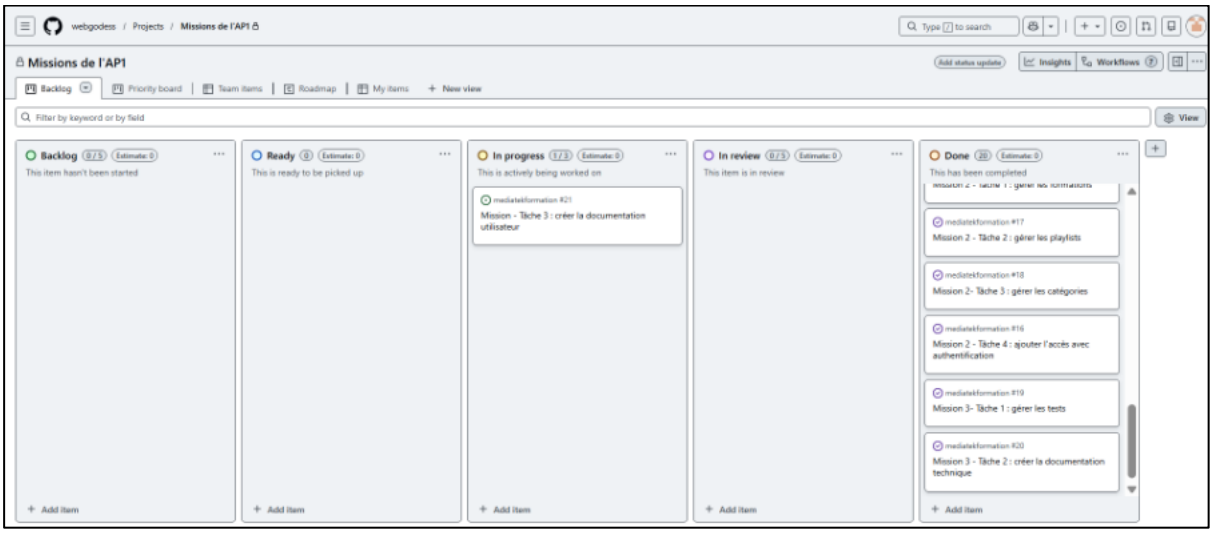
- préparation du parcours de démonstration ;
- sélection des fonctionnalités à présenter dans le front office ;
- sélection des fonctionnalités à présenter dans le back office ;
- enregistrement de la navigation dans le site ;
- ajout d'explications orales pour accompagner les actions réalisées ;
- vérification de la durée finale de la vidéo.

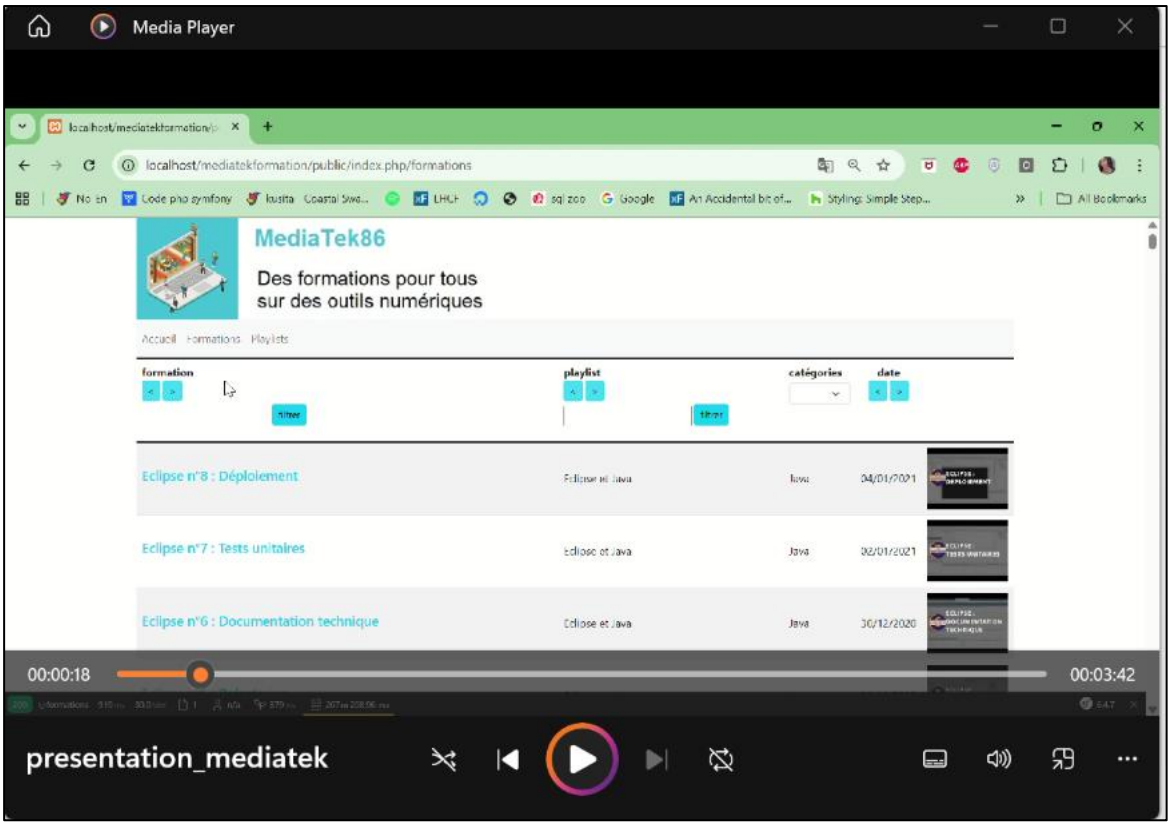
Résultat obtenu

La documentation utilisateur a été produite sous la forme d'une vidéo claire et synthétique. Elle permet de présenter le fonctionnement général du site et les principales actions réalisables dans le front office et le back office.

La vidéo constitue un support de présentation simple et accessible pour accompagner l'utilisation de l'application.

Captures d'écran





6. Mission 4 – Déploiement, sauvegarde et déploiement continu

6.1 Tâche 1 – Déployer le site

Demande de la tâche

Cette tâche consistait à déployer le site, la base de données et la documentation technique chez un hébergeur, puis à mettre à jour la page des CGU avec la bonne adresse du site.

Il fallait également vérifier que le site était opérationnel en ligne et que l'accès sécurisé à la partie administration fonctionnait correctement.

Temps estimé : **2 heures**

Temps réel : **3 heures**

Travail réalisé

Le site **MediatekFormation** a été déployé en ligne sur **Railway** à partir du dépôt GitHub du projet.
Une base de données MySQL a été mise en place et reliée à l'application Symfony.
La page des CGU a ensuite été mise à jour avec la bonne adresse du site ainsi qu'avec le bon hébergeur.

Le service a été configuré pour fonctionner en environnement de production avec les variables nécessaires au bon déploiement de Symfony, notamment :

- `APP_ENV=prod`
- `APP_SECRET=...`
- `COMPOSER_ALLOW_SUPERUSER=1`
- `RAILPACK_PHP_ROOT_DIR=/app/public`

La base de données a ensuite été remplie par import manuel d'un dump SQL existant via **MySQL Workbench**.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

- création d'un projet sur Railway et liaison avec le dépôt GitHub ;
- configuration du service de déploiement à partir de la branche utilisée pour le projet ;
- ajout des variables d'environnement nécessaires à Symfony ;
- mise en place d'une base MySQL ;
- import du dump SQL existant dans la base distante via MySQL Workbench ;
- vérification des logs de déploiement et de démarrage du service ;
- génération de l'URL publique du site ;
- mise à jour de la page des CGU avec la bonne adresse et le bon hébergeur ;
- test des pages `/login` et `/admin` afin de vérifier le bon fonctionnement de l'accès sécurisé.

Lors du premier déploiement, l'application ne fonctionnait pas correctement car l'extension PHP nécessaire à la connexion MySQL n'était pas chargée.
Le problème a été corrigé en ajoutant la variable suivante dans Railway :

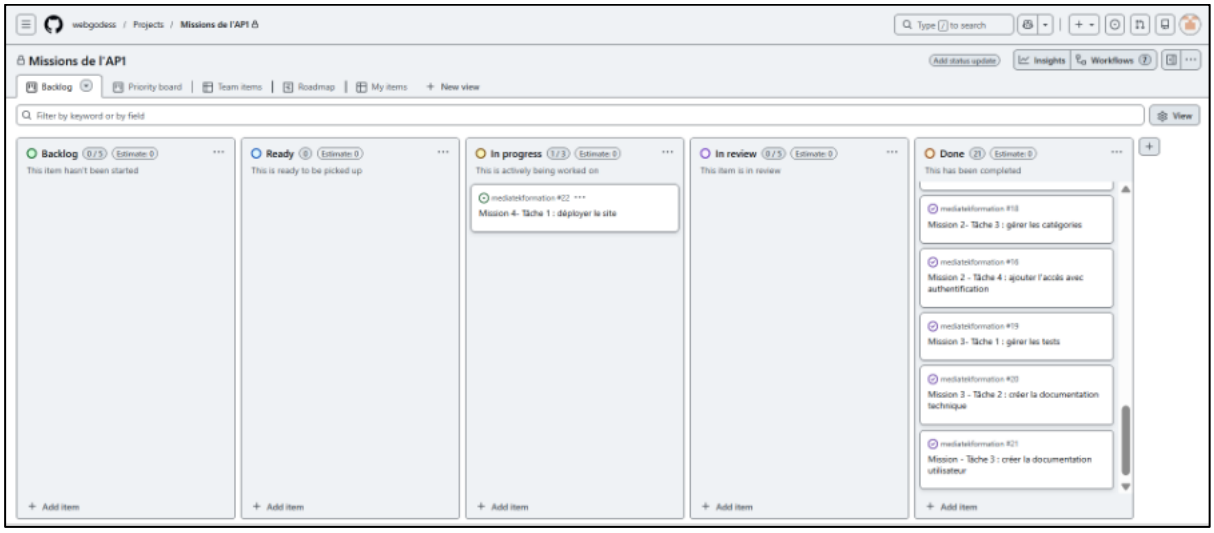
```
RAILPACK_PHP_EXTENSIONS=pdo_mysql
```

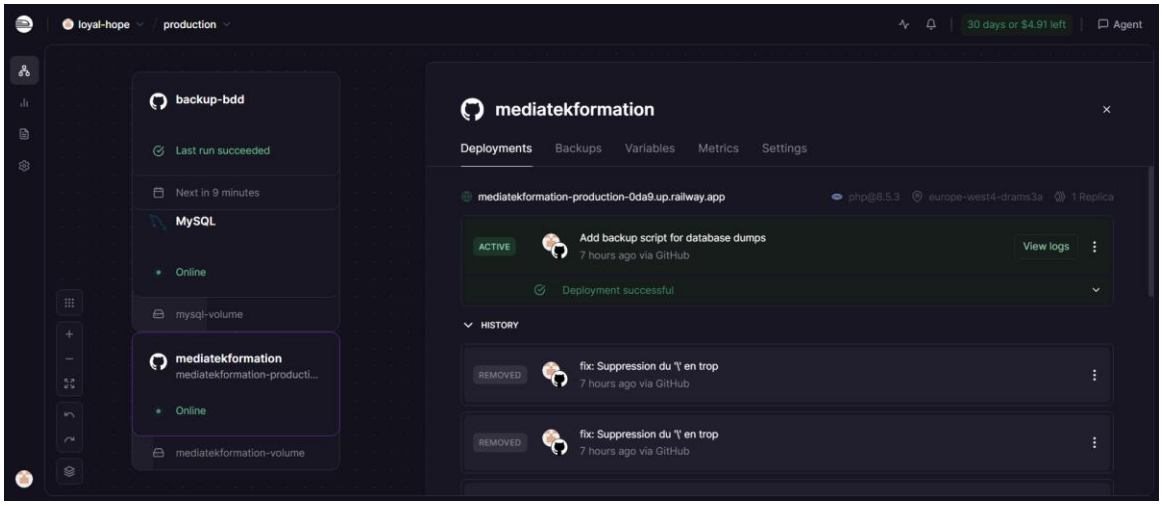
Cette correction a permis à Symfony et Doctrine de communiquer correctement avec la base MySQL.

Résultat obtenu

Le site a bien été mis en ligne sur Railway.
La base de données MySQL a été créée puis remplie à partir du dump SQL existant.
La page des CGU a été mise à jour avec la bonne adresse du site et le bon hébergeur.
Le site public fonctionne et l'accès sécurisé à la partie administration est opérationnel.

Captures d'écran





6.2 Tâche 2 – Gérer la sauvegarde et la restauration de la BDD

Demande de la tâche

Cette tâche consistait à mettre en place une sauvegarde journalière automatisée de la base de données et à prévoir une restauration manuelle à partir du fichier de sauvegarde.

Temps estimé : 1 heure

Temps réel : 1 h 05 min

Travail réalisé

Une solution de sauvegarde a été mise en place sur **Railway** pour la base de données MySQL de l'application. Un service dédié nommé **`backup-bdd`** a été créé afin d'exécuter automatiquement les sauvegardes, ainsi qu'un **volume persistant** monté sur le chemin **`/savebdd`** pour stocker les fichiers générés.

Un script shell nommé **`backup.sh`** a ensuite été ajouté au projet.
Ce script effectue un export de la base avec `mysqldump`, compresse le résultat au format `.sql.gz`, puis enregistre le fichier dans le dossier `/savebdd`.

Le script utilise les variables d'environnement fournies par Railway pour se connecter à la base MySQL :

- `MYSQLHOST`
- `MYSQLPORT`
- `MYSQLUSER`
- `MYSQLPASSWORD`
- `MYSQLDATABASE`

Le service `backup-bdd` a ensuite été configuré comme **cron job** sur Railway afin que la sauvegarde soit exécutée automatiquement chaque jour.
Un test manuel avec l'option **Run now** a permis de vérifier que la sauvegarde fonctionnait correctement.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

- création d'un volume persistant monté sur `/savebdd` ;
- création d'un service dédié nommé `backup-bdd` ;
- ajout du script `backup.sh` au projet ;
- configuration des variables nécessaires à la connexion MySQL ;
- configuration du service de sauvegarde pour une exécution automatique ;
- test manuel du fonctionnement de la sauvegarde avec l'option **Run now**.

Organisation de la sauvegarde

La sauvegarde repose sur trois éléments :

- un script `backup.sh` ;
- un service Railway dédié à la sauvegarde ;
- un volume persistant monté sur `/savebdd`.

Cette organisation permet de conserver les fichiers de sauvegarde de manière persistante et séparée du site principal.

Mode opératoire pour la restauration

La restauration se fait manuellement à partir du fichier de sauvegarde enregistré dans `/savebdd`.

Le script `backup.sh` crée un fichier du type :

`/savebdd/bddbackup_2026-03-11.sql.gz`

Ce fichier contient une copie compressée de la base.

Pour restaurer la base, il faut :

- récupérer le fichier de sauvegarde souhaité ;
- le décompresser ;
- réinjecter son contenu dans la base MySQL.

Exemple de commande de restauration :

```
gunzip < /savebdd/bddbackup_2026-03-11.sql.gz | mysql -h
"$MYSQLHOST" -P "$MYSQLPORT" -u "$MYSQLUSER" -p"$MYSQLPASSWORD"
"$MYSQLDATABASE"
```

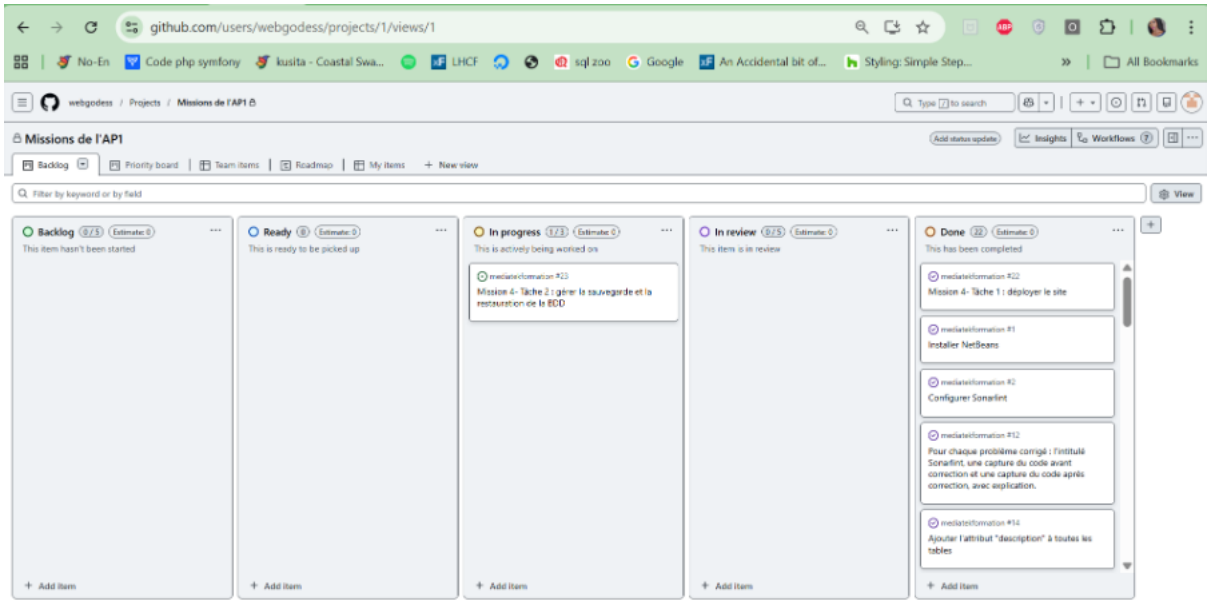
Cette commande :

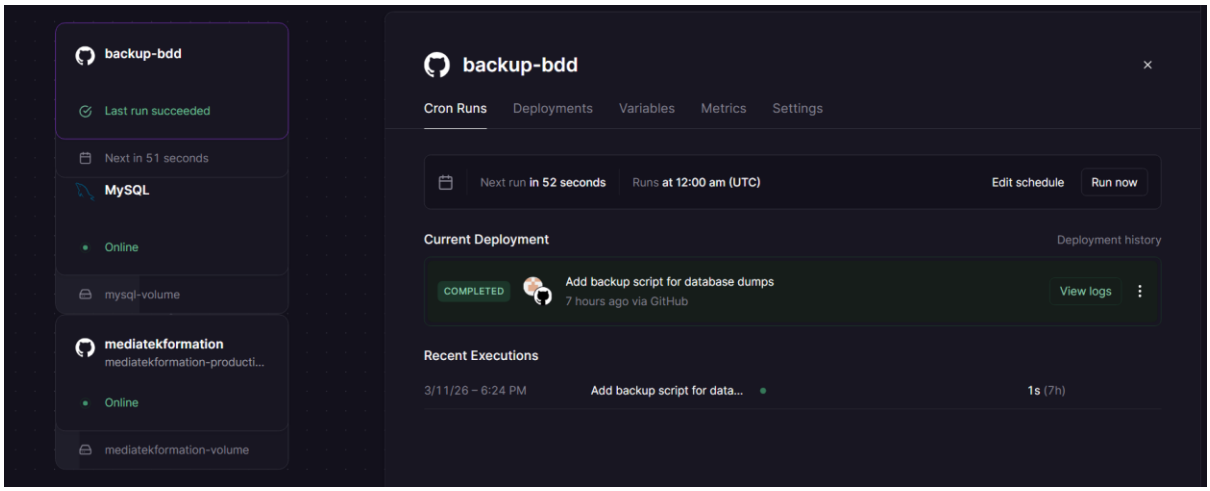
- décompresse la sauvegarde ;
- envoie le contenu décompressé à MySQL ;
- se connecte à la base distante ;
- réinjecte les données dans la base concernée.

Résultat obtenu

La sauvegarde journalière automatisée de la base de données a bien été programmée.
Le système de sauvegarde fonctionne sur Railway et la restauration manuelle reste possible à partir du fichier généré.

Captures d'écran





6.3 Tâche 3 – Mettre en place le déploiement continu

Demande de la tâche

Cette tâche consistait à configurer le dépôt GitHub afin que le site en ligne soit automatiquement mis à jour à chaque push reçu dans le dépôt.

Temps estimé : **1 heure**

Temps réel : **10 minutes**

Travail réalisé

Le déploiement continu a été mis en place entre le dépôt GitHub du projet et l'hébergement du site sur Railway.

Le service de production a été relié au dépôt GitHub du projet, puis la branche **`mods`** a été définie comme branche de déploiement.
Cette configuration permet à Railway de déclencher automatiquement un nouveau déploiement du site à chaque nouveau push effectué sur cette branche.

Une vérification a ensuite été réalisée pour s'assurer que le service était bien connecté au bon dépôt et à la bonne branche.
Après un nouveau push sur la branche `mods`, Railway a automatiquement lancé un nouveau build puis un nouveau déploiement, ce qui a confirmé le bon fonctionnement du déploiement continu.

Étapes suivies

Le travail a été réalisé en plusieurs étapes :

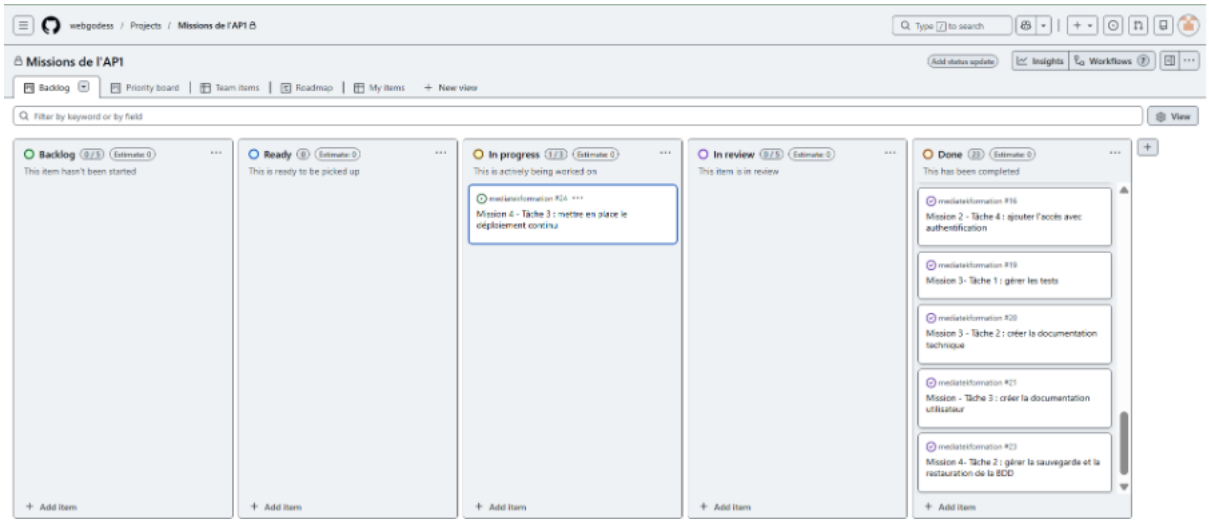
- connexion du service Railway au dépôt GitHub du projet ;
- vérification de la branche utilisée pour la production ;
- configuration de la branche `mods` comme branche de déploiement ;
- envoi d'un nouveau commit sur GitHub afin de tester le déclenchement automatique ;
- vérification dans Railway du lancement automatique du build et du déploiement ;
- contrôle final du site en ligne pour confirmer la prise en compte des modifications.

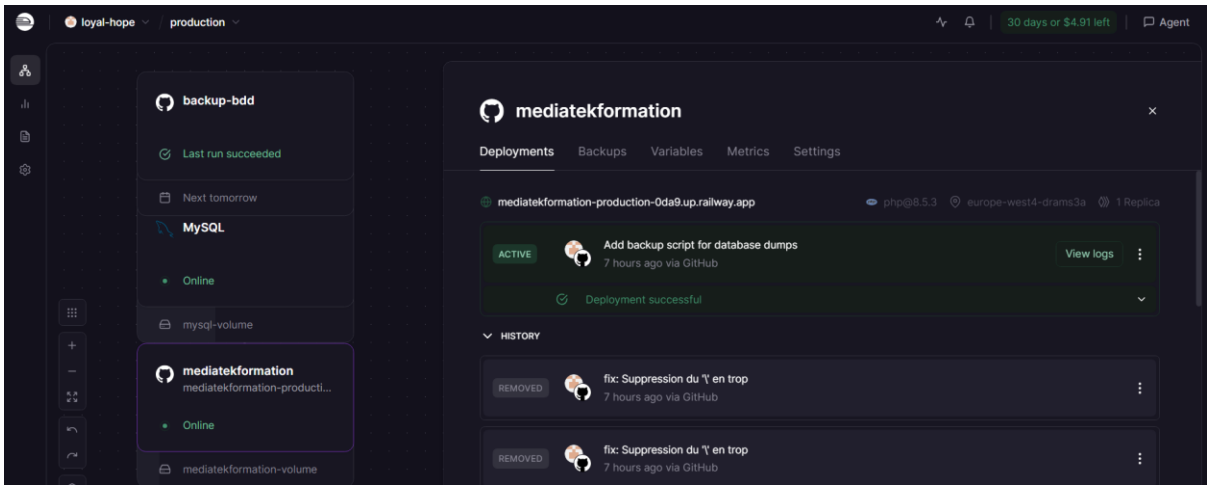
Le déploiement continu repose directement sur la liaison entre GitHub et Railway : lorsque des modifications sont poussées sur la branche `mods`, Railway détecte automatiquement le nouveau commit et redéploie le site.

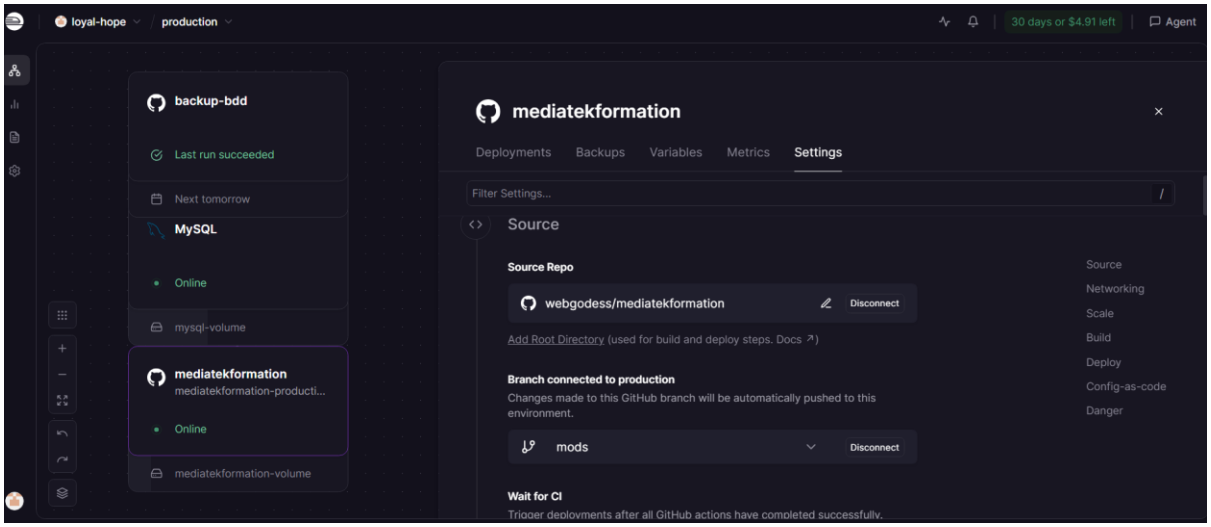
Résultat obtenu

Le déploiement continu est opérationnel.
Le site en ligne est désormais mis à jour automatiquement à chaque push effectué sur la branche `mods` du dépôt GitHub.

Captures d'écran







7. Bilan final

7.1 Objectifs atteints

L'atelier **AP1 - MediatekFormation** a permis d'atteindre les principaux objectifs fixés.
L'application a été reprise, modifiée, testée, documentée et mise en ligne.
Le site est désormais accessible en production, la base de données fonctionne correctement, l'accès sécurisé à la partie administration est opérationnel, une sauvegarde automatisée a été mise en place et le déploiement continu est actif.

7.2 Problèmes rencontrés

Plusieurs difficultés techniques ont été rencontrées au cours de l'atelier :

- compréhension et reprise d'un existant déjà structuré ;
- problèmes de déploiement en environnement de production ;
- erreur liée à l'extension MySQL manquante ;
- décalage entre la branche attendue et la branche réellement déployée ;
- adaptation de la procédure de sauvegarde à l'environnement Railway.

7.3 Solutions apportées

Les solutions mises en place ont permis de résoudre progressivement l'ensemble des problèmes rencontrés :

- vérification de la configuration Symfony et de la base MySQL ;
- ajout de la variable Railway `RAILPACK_PHP_EXTENSIONS=pdo_mysql` ;
- correction de la configuration de déploiement et vérification de la branche `mods` ;
- création d'un service dédié à la sauvegarde ;
- création d'un volume persistant sur `/savebdd` ;
- mise en place d'un cron job pour automatiser la sauvegarde.

7.4 Compétences mobilisées

Cet atelier a permis de mobiliser plusieurs compétences techniques et méthodologiques :

- reprise d'un projet existant ;
- développement web avec Symfony ;
- utilisation de Twig et de Doctrine ;
- gestion d'une base de données MySQL ;
- tests et validation technique ;
- rédaction de documentation ;
- gestion de versions avec Git et GitHub ;
- déploiement sur Railway ;
- administration technique d'un environnement de production ;
- mise en place d'une sauvegarde et d'un déploiement continu.

7.5 Conclusion générale

Cet atelier a permis de travailler sur un projet web dans des conditions proches d'un contexte réel, depuis l'analyse de l'existant jusqu'à la mise en ligne finale.
Il a également permis de consolider la compréhension du cycle complet de vie d'une application : développement, validation, documentation, déploiement, sauvegarde et maintenance.